Conferencia 10: divide y vencerás

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-11-18

La conferencia 8 terminó con una cuenta. La selección, la inserción y la burbuja multiplican su trabajo por cuatro cada vez que la lista se duplica, y con un millón de números tardarían horas. Y sin embargo `sorted` ordenaba ese mismo millón en menos de un segundo. Dijimos que `sorted` no era un n^2 escrito en un lenguaje más rápido, sino otro algoritmo.

Hoy escribimos dos algoritmos de esa otra clase, la **ordenación por mezcla** y la **ordenación rápida**, y los ponemos a ordenar el millón. Los dos usan la herramienta de la conferencia pasada, la función que se llama a sí misma más de una vez, y los dos siguen el mismo esquema, que tiene nombre: **divide y vencerás**. Partir el problema en pedazos, resolver cada pedazo con la misma función, y combinar las soluciones.

En los dos de hoy la división es evidente, porque una lista se parte en dos mitades. La semana próxima vemos problemas donde partir o combinar requiere pensar más.

1. Mezclar dos listas ordenadas

Empezamos por una pieza que no es recursiva. Tienes dos listas, cada una ya ordenada, y quieres una sola lista ordenada con todos los elementos de las dos.

Piensa en dos montones de exámenes, cada uno ordenado por nota de menor a mayor, boca arriba. El examen de nota más baja de todos es uno de los dos que están arriba: no puede estar debajo en un montón, porque ahí todos son mayores que el de arriba. Lo tomas, lo pones en el montón final, y repites con los dos que quedan arriba. Cuando un montón se acaba, el otro va entero al final, porque ya está ordenado.

```python
def mezclar(a, b):
    """Recibe dos listas ordenadas y devuelve una lista ordenada con todos sus elementos."""
    resultado = []
    i, j = 0, 0                # el de arriba de cada montón

    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            resultado.append(a[i])
            i += 1
        else:
            resultado.append(b[j])
            j += 1

    resultado.extend(a[i:])    # lo que quede de a, que ya está ordenado
    resultado.extend(b[j:])    # o lo que quede de b; uno de los dos está vacío
    return resultado

print(mezclar([2, 5, 8, 9], [1, 3, 4, 10, 12]))
```

```
[1, 2, 3, 4, 5, 8, 9, 10, 12]
```

Cada vuelta del `while` pone un elemento en `resultado`, así que hay como mucho `len(a) + len(b)` vueltas. **Mezclar cuesta tanto como la cantidad de elementos.** No hay un ciclo dentro de otro, como en la selección: cada elemento se mira una vez, se coloca, y no se vuelve a tocar.

Fíjate en el $\leqslant$ de la comparación. Con elementos iguales, se toma primero el de `a`. Eso hace que la mezcla conserve el orden de los iguales, la propiedad que en la conferencia 8 llamamos estabilidad.

2. Ordenación por mezcla

Ahora la idea. Si sabes mezclar dos listas ordenadas, ordenar una lista es:

1. Partirla en dos mitades.
2. Ordenar cada mitad.
3. Mezclar las dos mitades ordenadas.

El paso 2 es el mismo problema con la mitad de elementos. Ahí entra la recursión. El caso base es una lista de cero o un elemento, que ya está ordenada.

```
def ordenar_mezcla(lista):
    """Devuelve una lista nueva con los elementos de lista, ordenados."""
    if len(lista) <= 1:
        return list(lista)
    medio = len(lista) // 2
    izquierda = ordenar_mezcla(lista[:medio])
    derecha = ordenar_mezcla(lista[medio:])
    return mezclar(izquierda, derecha)

print(ordenar_mezcla([38, 27, 43, 3, 9, 82, 10, 5]))
```

```
[3, 5, 9, 10, 27, 38, 43, 82]
```

A diferencia de los métodos de la conferencia 8, esta función no modifica la lista que recibe: construye y devuelve una nueva, como `sorted`. Por eso el caso base devuelve `list(lista)`, una copia, y no `lista`: si devolviera la misma lista que recibió, con cero o un elemento el resultado y la original serían la misma lista con dos nombres, la sorpresa de la conferencia 4.

Lo primero, como siempre, es probarla contra algo que sabemos que está bien:

```
random.seed(1)
for prueba in range(500):
    n = random.randint(0, 12)
    original = [random.randint(1, 5) for _ in range(n)]
    if ordenar_mezcla(original) != sorted(original):
        print("FALLA", original)

print("pruebas terminadas")
```

```
pruebas terminadas
```

Para ver qué pasa por dentro, miramos el árbol de llamadas de la conferencia 9. Las llamadas parten la lista hacia abajo hasta llegar a listas de un elemento:

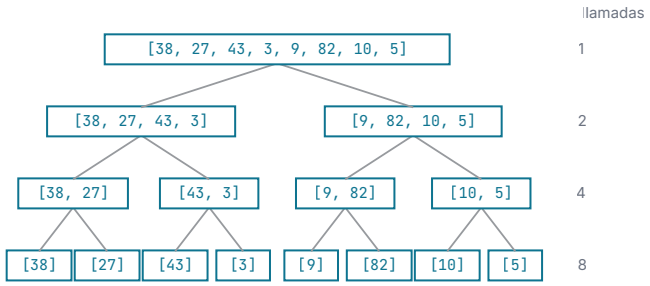


Figura 1. Cada llamada a `ordenar_mezcla` con la lista que recibe. Las llamadas parten la lista a la mitad hasta llegar a listas de un elemento, que ya están ordenadas. A la derecha, cuántas llamadas hay en cada nivel.

Y cuando las llamadas vuelven, cada una mezcla lo que le devolvieron sus dos hijas. El mismo árbol, ahora con lo que **devuelve** cada llamada:

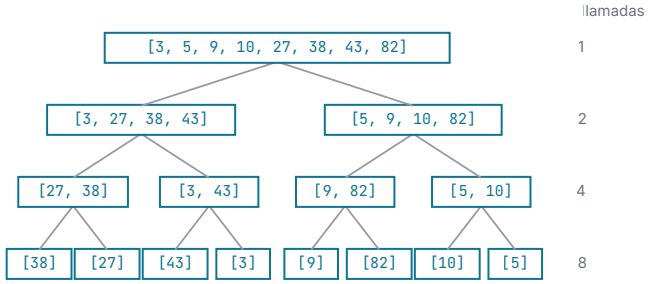


Figura 2. El mismo árbol que en la figura anterior, con lo que devuelve cada llamada. Se lee de abajo arriba: cada nodo es la mezcla de sus dos hijos.

3. Por qué es tan rápida

Contemos el trabajo con el árbol, nivel por nivel, como contamos los movimientos de Hanói en la conferencia 9.

En cada nivel se mezclan, en total, n elementos. En el nivel de arriba se mezclan dos listas de 4 en una de 8. En el siguiente, dos mezclas de dos listas de 2: otra vez 8 elementos en total. En el siguiente, cuatro mezclas de listas de 1: otra vez 8. Los pedazos se achican, pero hay más pedazos, y la suma es siempre la lista entera.

Hay $\log_2 n$ niveles. Cada nivel parte las listas a la mitad, y la pregunta de cuántas veces se puede partir n a la mitad hasta llegar a 1 es la de la búsqueda binaria: $\log_2 n$. Para 8 elementos las figuras tienen 4 niveles, pero el de abajo, el de las listas de un elemento, no mezcla nada: son 3 niveles de mezclas, y $\log_2 8 = 3$.

Entonces el trabajo total es n por nivel por $\log_2 n$ niveles: $n \log_2 n$. Lo comprobamos contando las comparaciones de `mezclar`:

```
comparaciones = 0

def mezclar_contando(a, b):
    global comparaciones
    resultado = []
    i, j = 0, 0
    while i < len(a) and j < len(b):
        comparaciones += 1
        if a[i] <= b[j]:
            resultado.append(a[i])
```

```
            i += 1
        else:
            resultado.append(b[j])
            j += 1
    resultado.extend(a[i:])
    resultado.extend(b[j:])
    return resultado

def ordenar_mezcla_contando(lista):
    if len(lista) <= 1:
        return lista
    medio = len(lista) // 2
    izquierda = ordenar_mezcla_contando(lista[:medio])
    derecha = ordenar_mezcla_contando(lista[medio:])
    return mezclar_contando(izquierda, derecha)

random.seed(8)
print(f"{'n':>6} {'mezcla':>8} {'n log n':>8} {'selección':>10}")
for n in [100, 200, 400, 800, 1600, 3200]:
    lista = [random.randint(1, 1_000_000) for _ in range(n)]
    comparaciones = 0
    ordenar_mezcla_contando(lista)
    print(f"{n:>6} {comparaciones:>8} {n * math.log2(n):>8.0f} {n * (n - 1) // 2:>10}")
```

```
     n   mezcla  n log n  selección
   100      545      664       4950
   200     1278     1529      19900
   400     2961     3458      79800
   800     6739     7715     319600
  1600    15047    17030    1279200
  3200    33290    37260    5118400
```

El `global comparaciones` le dice a Python que la función va a modificar la variable de afuera y no a crear una local con el mismo nombre. Es el error de alcance de la conferencia 3. Aquí lo usamos a propósito, porque el contador tiene que sobrevivir a todas las llamadas recursivas, pero no es una costumbre recomendable fuera de un experimento como este.

Lee la tabla por columnas. Cuando n se duplica, la selección multiplica sus comparaciones por cuatro, y la mezcla las multiplica por poco más de dos. Ese "poco más" es el $\log_2 n$, que sube de uno en uno cuando n se duplica. La mezcla hace un poco menos de $n \log_2 n$ comparaciones, porque cuando un montón se acaba el otro se copia sin comparar.

Para $n = 3200$, la selección hace más de cinco millones de comparaciones y la mezcla unas treinta y tres mil. Y la diferencia crece con n: es la diferencia entre multiplicar por n y multiplicar por $\log_2 n$, que para un millón es un millón contra unos veinte.

4. El esquema

La ordenación por mezcla tiene tres partes, y cualquier algoritmo de divide y vencerás las tiene:

- **Dividir** el problema en subproblemas del mismo tipo y más pequeños. Aquí, partir la lista por la mitad.
- **Vencer** cada subproblema con la misma función, recursivamente. Si es lo bastante pequeño, se resuelve directo: es el caso base.
- **Combinar** las soluciones de los subproblemas en la solución del problema. Aquí, mezclar.

La búsqueda binaria de la conferencia 9 también encaja, con una particularidad: divide la lista en dos mitades pero solo necesita resolver **una**, la que puede contener el valor, y combinar no cuesta nada,

porque la respuesta de la mitad es la respuesta. Por eso tiene $\log_2 n$ niveles con una sola llamada cada uno, en lugar de $\log_2 n$ niveles con n de trabajo cada uno.

Que un problema se pueda partir no basta para que divide y vencerás sea rápido. El costo total depende de cuánto cuestan las tres partes. En la mezcla, dividir es casi gratis y combinar cuesta n por nivel. El algoritmo siguiente hace lo contrario.

5. Ordenación rápida

La mezcla parte la lista sin mirarla y trabaja al combinar. La **ordenación rápida** trabaja al partir, para que combinar sea gratis.

Escoge un elemento de la lista, el **pivote**, y reparte los demás en tres listas: los menores que el pivote, los iguales y los mayores. Después ordena los menores y los mayores, recursivamente. Y ya está: todos los menores van antes que todos los iguales, que van antes que todos los mayores, así que basta pegar las tres listas.

```
def ordenar_rapida(lista):
    """Devuelve una lista nueva con los elementos de lista, ordenados."""
    if len(lista) <= 1:
        return list(lista)
    pivote = lista[0]
    menores, iguales, mayores = [], [], []
    for x in lista:
        if x < pivote:
            menores.append(x)
        elif x == pivote:
            iguales.append(x)
        else:
            mayores.append(x)
    return ordenar_rapida(menores) + iguales + ordenar_rapida(mayores)

print(ordenar_rapida([38, 27, 43, 3, 9, 82, 10, 5]))

random.seed(1)
for prueba in range(500):
    original = [random.randint(1, 5) for _ in range(random.randint(0, 12))]
    if ordenar_rapida(original) != sorted(original):
        print("FALLA", original)
print("pruebas terminadas")
```

```
[3, 5, 9, 10, 27, 38, 43, 82]
pruebas terminadas
```

El + entre listas las pega una detrás de otra, en una lista nueva. La lista `iguales` nunca está vacía, porque contiene al menos el pivote, y por eso las dos llamadas recursivas reciben listas más pequeñas que la original. Si pusiéramos el pivote junto con los menores, habría listas que se llamarían a sí mismas con la misma lista una y otra vez, hasta chocar con el tope de mil llamadas de la conferencia 9 y dar `RecursionError`; el ejercicio 4 te pide encontrarlas.

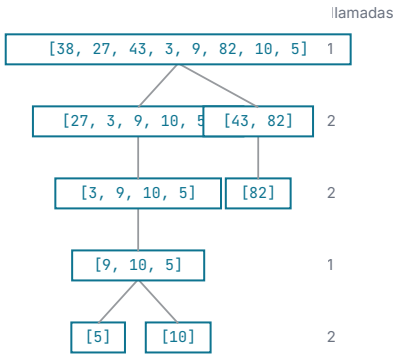


Figura 3. Las llamadas de `ordenar_rapida` sobre la misma lista. El pivote es el primer elemento de cada lista, y no aparece en los hijos porque se queda en `iguales`. No se dibujan las llamadas con la lista vacía.

Compara la figura 3 con la figura 1. La mezcla parte siempre exactamente a la mitad. La rápida parte donde caiga el pivote: el 38 deja cinco elementos a la izquierda y dos a la derecha. Si el pivote cae cerca del medio, el árbol tiene unos $\log_2 n$ niveles y en cada nivel se reparten, en total, unos n elementos: otra vez $n \log_2 n$. Pero eso depende de dónde caiga el pivote.

6. El pivote

¿Qué pasa si el pivote cae siempre en un extremo? Con el primer elemento como pivote, basta darle una lista **ya ordenada**. El primero es el menor, `menores` queda vacía, y `mayores` tiene todos los demás. Cada nivel quita un solo elemento:

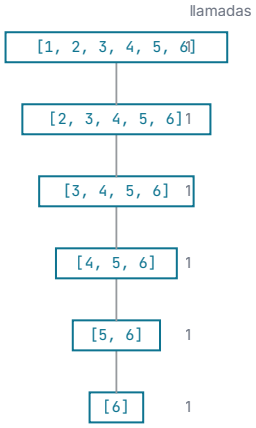


Figura 4. `ordenar_rapida([1, 2, 3, 4, 5, 6])`. El pivote es siempre el menor, y cada llamada le pasa a la siguiente la lista con un elemento menos. El árbol es una fila, como la pila del factorial.

El árbol de la figura 4 tiene n niveles en lugar de $\log_2 n$, y el nivel k reparte $n - k$ elementos. En total, $n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}$, casi la misma suma que la de la selección y del mismo orden: otra vez cuadrática. Y peor aún, con n niveles de recursión se llega al tope de la pila de la conferencia 9:

```
ordenada = list(range(2000))
try:
    ordenar_rapida(ordenada)
except RecursionError as error:
    print(type(error).__name__, error)
```

```
RecursionError maximum recursion depth exceeded
```

La lista ya ordenada es justo el caso que más se da en la práctica: datos que alguien ya ordenó, o que llegan casi en orden. Un algoritmo de ordenación que se rompe con una lista ordenada no sirve.

El arreglo es escoger el pivote **al azar**. `random.choice(lista)` devuelve un elemento cualquiera de la lista:

```python
def ordenar_rapida_azar(lista):
    if len(lista) <= 1:
        return list(lista)
    pivote = random.choice(lista)
    menores, iguales, mayores = [], [], []
    for x in lista:
        if x < pivote:
            menores.append(x)
        elif x == pivote:
            iguales.append(x)
        else:
            mayores.append(x)
    return ordenar_rapida_azar(menores) + iguales + ordenar_rapida_azar(mayores)

print(ordenar_rapida_azar(ordenada) == ordenada)
```

```
True
```

Ahora no existe ninguna lista que sea mala siempre. El pivote cae en un extremo con probabilidad pequeña, y para que el árbol quede como una fila tendría que caer en un extremo en casi todas las llamadas, lo que con listas grandes no pasa en la práctica. Lo que se puede demostrar, y se demuestra en Diseño y Análisis de Algoritmos, es que el trabajo **esperado** es proporcional a $n \log n$ con cualquier lista de entrada.

7. El reloj

Ahora sí, medimos. Primero contra los métodos de la conferencia 8, con las mismas listas y los mismos tamaños:

```python
def cronometrar(metodo, lista):
    copia = list(lista)
    inicio = time.perf_counter()
    metodo(copia)
    return time.perf_counter() - inicio

random.seed(8)
tamanos = [500, 1000, 2000, 4000]
tiempos = {"selección": [], "inserción": [], "mezcla": [], "rápida": []}
for n in tamanos:
    lista = [random.randint(1, 1_000_000) for _ in range(n)]
    tiempos["selección"].append(cronometrar(seleccion, lista))
    tiempos["inserción"].append(cronometrar(insercion, lista))
    tiempos["mezcla"].append(cronometrar(ordenar_mezcla, lista))
    tiempos["rápida"].append(cronometrar(ordenar_rapida_azar, lista))

print(f"{'n':>5}" + "".join(f"{m:>11}" for m in tiempos))
for k, n in enumerate(tamanos):
    print(f"{n:>5}" + "".join(f"{tiempos[m][k]:>10.3f}s" for m in tiempos))
```

```
   n  selección  inserción     mezcla     rápida
 500     0.005s     0.004s     0.001s     0.001s
1000     0.020s     0.019s     0.002s     0.001s
2000     0.081s     0.079s     0.004s     0.003s
4000     0.310s     0.334s     0.020s     0.006s
```

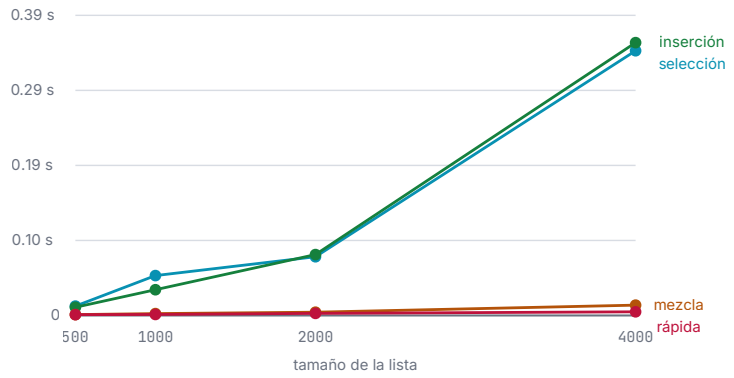


Figura 5. Los segundos de la tabla anterior. Las dos curvas de la conferencia 8 se doblan hacia arriba; la mezcla y la rápida se quedan pegadas al fondo.

En la figura 5, la mezcla y la rápida apenas se separan del cero. Con 4000 elementos ya son más de diez veces más rápidas, y la ventaja crece con cada duplicación.

Y ahora, el millón que en la conferencia 8 habría tardado horas:

```python
random.seed(8)
millon = [random.randint(1, 1_000_000) for _ in range(1_000_000)]

for nombre, metodo in [("mezcla", ordenar_mezcla), ("rápida", ordenar_rapida_azar),
                       ("sorted", sorted)]:
    inicio = time.perf_counter()
    resultado = metodo(millon)
    print(f"{nombre:>7}: {time.perf_counter() - inicio:5.2f} s  "
          f"{'bien' if resultado == sorted(millon) else 'MAL'}")
```

```
 mezcla:  4.31 s  bien
 rápida:  3.01 s  bien
 sorted:  0.44 s  bien
```

Segundos en lugar de horas, y escrito en Python, con listas y `append`, sin ningún truco. Lo que cambió no es la máquina ni el lenguaje: es el orden de crecimiento.

`sorted` sigue siendo varias veces más rápida que las nuestras, por dos razones. Está escrita en C, no en Python, y eso le da una constante mucho menor. Y su algoritmo, que se llama Timsort, es una ordenación por mezcla con mejoras: aprovecha los pedazos que ya vienen ordenados en la entrada y usa la inserción para los pedazos pequeños, donde es más rápida que la recursión. Pero es de la misma familia que la que escribimos hoy, y su orden de crecimiento es el mismo.

8. Resumen

- **Divide y vencerás**: dividir el problema en subproblemas más pequeños del mismo tipo, resolverlos recursivamente, y combinar sus soluciones.

- **Mezclar** dos listas ordenadas cuesta tanto como la cantidad de elementos: cada elemento se mira una vez.
- La **ordenación por mezcla** parte la lista a la mitad sin mirarla, ordena cada mitad y mezcla. Hace $\log_2 n$ niveles con n de trabajo cada uno: $n \log_2 n$. Es estable.
- La **ordenación rápida** reparte la lista en menores, iguales y mayores que un pivote, ordena los menores y los mayores, y pega. Trabaja al partir, y combinar es gratis.
- Con el primer elemento como pivote, una lista ya ordenada deja el árbol como una fila: trabajo cuadrático y `RecursionError` con listas de algunos miles. Con el pivote al azar, el trabajo esperado es $n \log n$ con cualquier entrada.
- Contar el trabajo de un algoritmo recursivo es contar el árbol de llamadas: cuántos niveles tiene, y cuánto trabajo hay en cada nivel.
- La búsqueda binaria es divide y vencerás con un solo subproblema.
- Pasar de n^2 a $n \log n$ es pasar de horas a segundos con un millón de elementos.

Ejercicios

1. Escribe `mezclar_tres(a, b, c)` que mezcle tres listas ordenadas en una. Hazlo de dos maneras: llamando dos veces a `mezclar`, y con un solo ciclo que mire los tres elementos de arriba. ¿Cuál hace menos comparaciones?
2. Modifica `ordenar_mezcla` para que parta la lista en **tres** pedazos en lugar de dos, usando el ejercicio anterior. ¿Cuántos niveles tiene el árbol? ¿Es más rápida? Mídelo.
3. Escribe `ordenar_mezcla_ciclo(lista)` **sin recursión**: empieza con cada elemento como una lista de uno, y mezcla de dos en dos hasta que quede una sola. Es el árbol de la figura 2 recorrido de abajo arriba.
4. Cambia `ordenar_rapida` para que el pivote vaya en `menores` en lugar de en `iguales`, es decir, usando `x ⩽ pivote`. Encuentra la lista más corta con la que la función dé `RecursionError`, y explica qué parte del razonamiento de la sección 5 deja de ser cierta.
5. Escribe `contar_rapida(lista)` que cuente las comparaciones de `ordenar_rapida` con el primer elemento como pivote, y úsala con listas ordenadas de 100, 200 y 400 elementos. ¿Se parece a la suma $\frac{n(n+1)}{2}$ de la sección 6? ¿Por qué no hace falta que la lista esté ordenada de menor a mayor para el peor caso? Encuentra otra lista mala.
6. Usa `ordenar_mezcla` para ordenar las parejas `(palabra, veces)` del Quijote de la conferencia 8 de más a menos frecuente. Tienes que cambiar la comparación en `mezclar`. Comprueba que las palabras con el mismo conteo quedan en el mismo orden que con la inserción de la conferencia 8.
7. La **mediana** de una lista es el elemento que quedaría en el medio si se ordenara. Escribe `seleccionar(lista, k)` que devuelva el elemento que quedaría en la posición `k`, usando la idea de la rápida pero con **una sola** llamada recursiva: después de repartir, sabes en cuál de las tres listas está la posición `k`. ¿Cuánto trabajo hace, si el pivote parte siempre más o menos a la mitad?
8. Con un millón de elementos, la ordenación rápida con el primer elemento como pivote funciona bien si la lista está al azar. Calcula cuántos niveles tendría el árbol si la lista estuviera ordenada, y cuánto tiempo tardaría, usando lo que mediste en la conferencia 8 para un algoritmo cuadrático. En Python, con el tope de mil llamadas, ¿qué pasa en realidad: termina, o da `RecursionError`? ¿Con qué tamaño de lista empieza a fallar?